

Lekcja

Temat: Asynchroniczność, synchroniczność, event loop w Pythonie

Synchroniczność oznacza wykonywanie instrukcji **jedna po drugiej** — program czeka aż dana operacja się zakończy, zanim przejdzie dalej.

```
import time

print("Start")
time.sleep(3)
print("Koniec")
```

Asynchroniczność pozwala wykonywać inne zadania w czasie oczekiwania na zakończenie operacji (np. pobierania danych z internetu, odczytu pliku, zapytania do bazy).

```
import asyncio

async def zadanie():
    print("Start")
    await asyncio.sleep(3)
    print("Koniec")
```

```
asyncio.run(zadanie())
```

Główne narzędzia:

- **Coroutines** — funkcje, które mogą być wstrzymywane i wznawiane (**async def**)
- **async/await** — składnia do pracy z coroutine'ami
- **asyncio** — pętla zdarzeń (**event loop**), która zarządza wszystkim

Przykład: **asyncio.sleep**

```
import asyncio
import time

def sync_sleep():
    print("Zaczynam spać sync...")
    time.sleep(2)
    print("Obudziłem się sync")
```

```
async def async_sleep():
    print("Zaczynam spać async...")
    await asyncio.sleep(2)
    print("Obudziłem się async")
```

```
async def main():
    await async_sleep()
```

```
asyncio.run(main())
```

Instalacja pip i aiohttp

1. Otwórz terminal / wiersz polecenia i wpisz:
python -m ensurepip --upgrade
python -m pip install aiohttp

Przykład

```
import asyncio
import aiohttp
```

```
async def main():
    url = "https://jsonplaceholder.typicode.com/users"

    async with aiohttp.ClientSession() as session:
        async with session.get(url) as response:
            print(f"Status odpowiedzi: {response.status}")

            users = await response.json()
            print(f"\nPobrano {len(users)} użytkowników\n")

            for user in users[:3]:
                print(f"• {user['name']}")
                print(f"  Email: {user['email']}")
                print(f"  Firma: {user['company']['name']}")
                print("-" * 40)
```

```
asyncio.run(main())
```

1. Coroutines (Korutyny) — async def

Coroutine to specjalna funkcja, którą można **wstrzymać** (await) i później **wznowić** w dowolnym momencie. Dzięki temu event loop może w tym czasie robić inne rzeczy.

```

import asyncio
import time

# Zwykła funkcja (synchronous)
def zwykla_funkcja():
    print("Zaczynam pracę")
    time.sleep(2)
    print("Skończyłem")

# Coroutine (asynchroniczna funkcja)
async def coroutine_przyklad():
    print("Zaczynam coroutine...")
    await asyncio.sleep(2)      # ← tu coroutine się wstrzymuje
    print("Coroutine wznowiona po 2 sekundach")
    return "Zakończono!"

# Uruchomienie
async def main():
    coro = coroutine_przyklad()    # ← tworzy coroutine (jeszcze nie działa!)
    print(type(coro))              # <class 'coroutine'>

    wynik = await coro             # ← teraz się wykonuje
    print(wynik)

asyncio.run(main())

```

Kluczowe cechy coroutines:

- Tworzy się przez `async def`
- Nie wykonuje się od razu po wywołaniu
- Musi być uruchomiona przez `await` lub `asyncio.run()`

2. **async/await** — składnia do pracy z coroutine'ami

`async` i `await` to **słowa kluczowe**, które razem tworzą asynchroniczny kod.

```

import asyncio

async def pobierz_dane():
    print("Zaczynam pobieranie...")
    await asyncio.sleep(1.5)      # await = wstrzymaj i oddaj kontrolę

```

```

    print("Dane pobrane!")
    return {"id": 1, "nazwa": "Jan"}

async def main():
    print("Start programu")

    # await = czekaj na wynik tej coroutiney
    dane = await pobierz_dane()

    print("Otrzymałem:", dane)
    print("Koniec programu")

asyncio.run(main())

```

Reguły:

- await można używać **tylko** wewnątrz async def
- await może czekać na inną coroutine, Task, lub obiekt z metodą `__await__()`

3. **asyncio** — pętla zdarzeń (Event Loop)

To jest **serce** całej asynchroniczności w Pythonie. Event Loop zarządza wszystkimi coroutine'ami, decyduje kiedy którą wznowić, obsługuje timer'y, I/O itp.

```

import asyncio

async def zadanie1():
    await asyncio.sleep(1)
    print("Zadanie 1 zakończone")

async def zadanie2():
    await asyncio.sleep(2)
    print("Zadanie 2 zakończone")

async def main():
    print("Event Loop zaczyna działać...")

    # Uruchamiamy wiele zadań "równolegle"
    await asyncio.gather(
        zadanie1(),
        zadanie2()
    )

    print("Event Loop zakończył wszystkie zadania")

```

```
# asyncio.run() → tworzy event loop, uruchamia main() i go zamyka
asyncio.run(main())
```

Porównanie wszystkich trzech razem

```
import asyncio
```

```
# 1. Coroutine
```

```
async def pobierz(url):
    print(f"[{url}] Rozpocynam pobieranie...")
    await asyncio.sleep(2)          # symulacja opóźnienia sieci
    print(f"[{url}] Zakończono!")
    return f"dane z {url}"
```

```
# 2. async/await + 3. asyncio
```

```
async def main():
    # Tworzymy listę coroutine'ów
    coro1 = pobierz("https://api1.com")
    coro2 = pobierz("https://api2.com")
    coro3 = pobierz("https://api3.com")

    # Event Loop zarządza wykonaniem
    wyniki = await asyncio.gather(coro1, coro2, coro3)

    print("Wszystkie wyniki:", wyniki)
```

```
asyncio.run(main())
```

Event Loop w Pythonie to mechanizm, który zarządza zadaniami asynchronicznymi (async/await).

Można powiedzieć, że to:

„koordynator”, który pilnuje kiedy dane zadanie ma się wykonywać i kiedy program może przełączyć się na inne zadanie.

Najczęściej używa się go przez bibliotekę asyncio.

Event Loop:

1. uruchamia zadania (async def)
2. sprawdza czy jakieś zadanie czeka (await)
3. jeśli czeka → przełącza się na inne zadanie

4. wraca do zadania gdy operacja się zakończy

Przykład

```
import asyncio

async def zadanie1():
    print("Start zadanie1")

    await asyncio.sleep(2)

    print("Koniec zadanie1")

async def zadanie2():
    print("Start zadanie2")

    await asyncio.sleep(1)

    print("Koniec zadanie2")

async def main():
    await asyncio.gather(
        zadanie1(),
        zadanie2()
    )

asyncio.run(main())
```

synchronicznie

```
import time

def zadanie1():
    print("Start 1")
    time.sleep(2)
    print("Koniec 1")

def zadanie2():
    print("Start 2")
    time.sleep(1)
    print("Koniec 2")
```

```
zadanie1()
zadanie2()
```

Wersja **SYNCHRONICZNA** (requests)

Potrzebana instalacja

```
python -m pip install requests
```

```
import requests
import time

def pobierz_dane_sync():
    urls = [
        "https://jsonplaceholder.typicode.com/users",
        "https://jsonplaceholder.typicode.com/posts",
        "https://jsonplaceholder.typicode.com/albums",
        "https://jsonplaceholder.typicode.com/todos",
    ]

    wyniki = []
    start = time.time()

    for url in urls:
        print(f"Pobieram: {url}")
        response = requests.get(url)          # ← blokuje program
        dane = response.json()
        wyniki.append(len(dane))
        print(f"✓ Pobrano {len(dane)} rekordów")

    czas = time.time() - start
    print(f"\nSynchronicznie zakończono w: {czas:.2f} sekund")

pobierz_dane_sync()
```

Charakterystyka:

- Kod jest prostszy i łatwiejszy do zrozumienia
- Zapytania wykonują się **po kolei** (kolejno)
- Cały program czeka na każde zapytanie

Wersja ASYNCHRONICZNA (aiohttp)

```
import asyncio
import aiohttp
import time

async def pobierz(url: str, session: aiohttp.ClientSession):
    async with session.get(url) as response:
        dane = await response.json()
        print(f"✓ Pobrano {len(dane)} rekordów z {url.split('/')[-1]}")
        return len(dane)

async def main():
    urls = [
        "https://jsonplaceholder.typicode.com/users",
        "https://jsonplaceholder.typicode.com/posts",
        "https://jsonplaceholder.typicode.com/albums",
        "https://jsonplaceholder.typicode.com/todos",
    ]

    start = time.time()

    async with aiohttp.ClientSession() as session:
        # Uruchamiamy wszystkie zapytania równolegle
        zadania = [pobierz(url, session) for url in urls]
        wyniki = await asyncio.gather(*zadania)

    czas = time.time() - start
    print(f"\nAsynchronicznie zakończono w: {czas:.2f} sekund")

asyncio.run(main())
```